AEDS 2 — Lista de questões das Provas II antigas

PUC Minas · Algoritmos e Estruturas de Dados II · Transcrição das provas antigas, separada por assunto

Como usar esta lista

- Os enunciados foram transcritos exatamente como estão nas provas. As figuras foram recortadas das fotos originais.
- A etiqueta à direita de cada questão indica de qual prova ela veio (versões numeradas de 1 a 7). Para fazer um simulado, junte as questões da mesma versão.
- Instruções que aparecem em uma das provas: *“Para as questões que exigem implementação, utilize exclusivamente o código fornecido como base. Todo o desenvolvimento adicional necessário para resolver a questão deve ser incluído e detalhado na sua resposta final.”*

1. Listas encadeadas

1.1 — Meiose

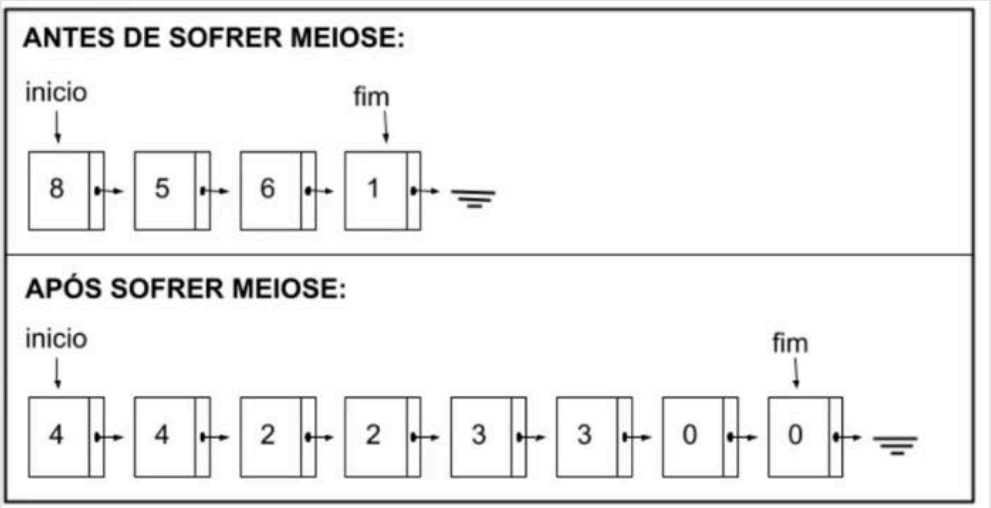
Prova II · versão 2 · Questão 1

Dada uma lista simplesmente encadeada, implemente a função ***void meiose()***. Esta função deve percorrer toda a lista, dividindo cada célula em duas, com cada nova célula contendo a metade do valor da célula original. Abaixo, é apresentada a estrutura do código e uma ilustração que mostra a lista antes e após a operação. Apresente a complexidade do código em função de theta.

```
typedef struct Celula {
int elemento;
struct Celula* prox;
} Celula;

Celula* inicio;
Celula* fim;

void Meiose(){
   //implemente a função aqui
   }
```



> Em uma das cópias da prova, os ponteiros globais se chamam `primeiro` e `ultimo` em vez de `inicio` e `fim`.

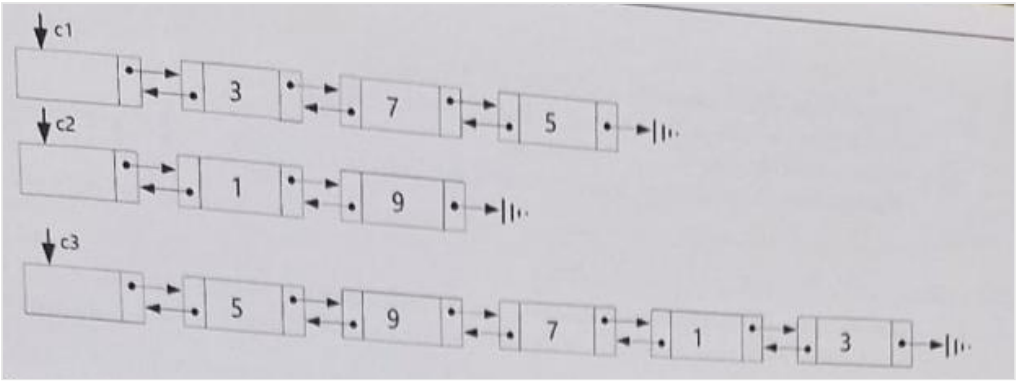
1.2 — Intercalação reversa

Prova II · versão 3 · Questão 3

Implemente um método que receba o nó cabeça de duas listas duplamente encadeadas e retorne o nó cabeça de uma nova lista, resultante da intercalação reversa dos elementos das duas listas, ou seja, partindo do último para o primeiro elemento de cada lista. A nova lista deve ser construída intercalando um elemento do final de uma lista com um elemento do final da outra, alternadamente.

Considere o código a seguir como base para a implementação. Para facilitar a compreensão, observe o exemplo gráfico apresentado.

```
class CelulaDupla {
    int elemento;
    CelulaDupla prox, ant;
}
CelulaDupla intercalarReverso(CelulaDupla c1, CelulaDupla c2) {
    //IMPLEMENTAR
}
```



1.3 — Remoção de item com *equals*

Prova II · versão 1 · Questão 3 (5 pontos)

Seja a classe **Lista<E>** simplesmente encadeada, vista em nossas aulas, e seus métodos básicos, conforme abaixo. Acrescente à classe Lista<E> um método capaz de **localizar e remover** um determinado item, passado como parâmetro. Se o elemento não se encontrar na lista ou se a mesma estiver vazia, seu método deverá lançar uma exceção. Seu método deve **obrigatoriamente** ter a assinatura ***public E remocaoDoItem(E item)***. Utilize obrigatoriamente o método **equals** para a comparação entre os itens da lista.

```
public class Lista<E> {

    private Celula<E> primeiro;
    private Celula<E> ultimo;
    private int tamanho;

    public boolean vazia() {...}

    public void inserir(E novo, int posicao) {...}

    public E remover(int posicao) {...}
}
```

1.4 — Quicksort em lista duplamente encadeada circular

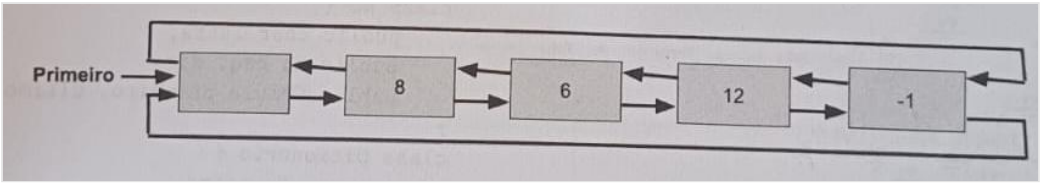
Prova II · versão 4 · Questão 3

Implemente a função void quicksort() em linguagem C para uma Lista Duplamente Encadeada Circular com Célula Cabeça, que deverá, através do algoritmo de ordenação por quicksort, ordenar os itens da lista. A lista é globalmente representada apenas pelo ponteiro primeiro, que aponta para a célula cabeça (sentinela); a célula cabeça não armazena dados válidos. A lista é circular, ou seja, o ponteiro ant da célula cabeça aponta para a última célula da lista, e o ponteiro prox da última célula aponta para a célula cabeça. Apresente, em função de θ, a complexidade do seu método.

```
typedef struct CelulaDupla {
    int elemento;              // Elemento inserido na celula.
    struct CelulaDupla* prox;  // Aponta a celula prox.
    struct CelulaDupla* ant;   // Aponta a celula anterior.
} CelulaDupla;

//LISTA PROPRIAMENTE DITA ========================================
CelulaDupla* primeiro;

void quicksort() {
    //implementar aqui
}
```



2. Pilhas e filas

2.1 — Notação polonesa reversa (pós-fixa)

Prova II · versão 1 · Questão 2 (4 pontos)

Calculadoras especiais de Engenharia usam a notação polonesa reversa (pós-fixa) para realizar cálculos matemáticos devido à eficiência de sua execução com o uso de pilhas.

a) Dada a expressão matemática abaixo, em notação infixa, qual seria sua representação em notação pós-fixa?

```
((a+b)^(c-d*e)+f)*(g/(h-i^j))          Notação pós-fixa: ____________________
```

b) E qual seria o resultado da conta denotada pela expressão matemática abaixo que já está em notação pós-fixa? Considere a=1, b=2, c=3, d=4, e=5, f=6, g=7, h=8, i=9, j=2

```
a b c d e f - ^ + * + g h + i j - * /          Resultado: ________
```

2.2 — Fila dupla (duas filas + uma pilha)

Prova II · versão 1 · Questão 4 (6 pontos)

Suponha que você trabalhe na Matriz de uma empresa que vende seus produtos em filiais espalhadas pelo estado. Os pedidos recebidos na matriz, em estruturas de dados do tipo **Fila flexível**, possuem os produtos que precisam ser produzidos para entrega. A nova abordagem de negócios da empresa quer que seja possível **inserir** um pedido na **frente da fila** de pedidos quando se tratar de um pedido urgente. Também é desejado

que seja possível a **remoção** de um pedido que esteja **no fim da fila** de pedidos. Sua tarefa então é **completar** a classe **FilaDupla<E>** abaixo, implementando dois métodos: um método que seja capaz de remover um item do fim da fila, com assinatura ***T item removerItemFim ()*** e outro método que possibilite inserir um item no começo da fila, com assinatura ***void inserirComeco(T item).***

Atenção: Use **obrigatoriamente** a estrutura de classe FilaDupla<E> abaixo, composta por duas filas e uma pilha. Os métodos com {...} estão devidamente implementados e se comportam como os vistos em nossas aulas.

```
public class Celula<T> {
  private final T item;
  private Celula<T> proximo;

  public T getItem() {...}
  public Celula<T> getProximo(){...}
  public void setProximo(Celula<T>
proximo){...}
}
```

```
public class Pilha<E> {
  public Pilha(){...}
  public boolean vazia(){...}
  public void empilhar(E item){...}
  public E desempilhar(){...}
  public E consultarTopo(){...}
}
```

```
public class FilaDupla<E> {
   private Fila<E> filaPrincipal;
   private Fila<E> filaAux;
   private Pilha<E> pilha;

   FilaDupla( ) { filaPrincipal = new Fila<>(); filaAux = new Fila<>(); pilha = new
Pilha<>();}

   public boolean vazia() { return filaPrincipal.vazia();}
   public void enfileirarFim(E item) {
       filaPrincipal.enfileirar(E item);
   }
   public E desenfileirarInicio() {
       return filaPrincipal.desenfileirar();
   }
   public void enfileirarInicio(E item) {

   }
   public E desenfileirarFim( ) {

   }
}
```

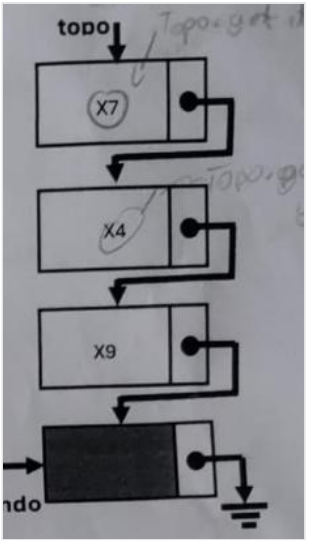
> Na prova, os métodos a completar no código se chamam `enfileirarInicio` e `desenfileirarFim`, enquanto o texto do enunciado usa os nomes `inserirComeco` e `removerItemFim`. A transcrição mantém os dois como estão na prova.

2.3 — Imprimir pilha em ordem reversa

Prova II · versão 1 · Questão 6 (4 pontos)

Implemente, na classe Pilha<E> simplesmente encadeada vista em nossas aulas, um método em Java, para imprimir, **em ordem reversa**, os itens existentes na pilha. O método deve ter a assinatura ***public void imprimirReverso ()***.

Neste exemplo ao lado, seu método imprimiria X9, X4, X7.

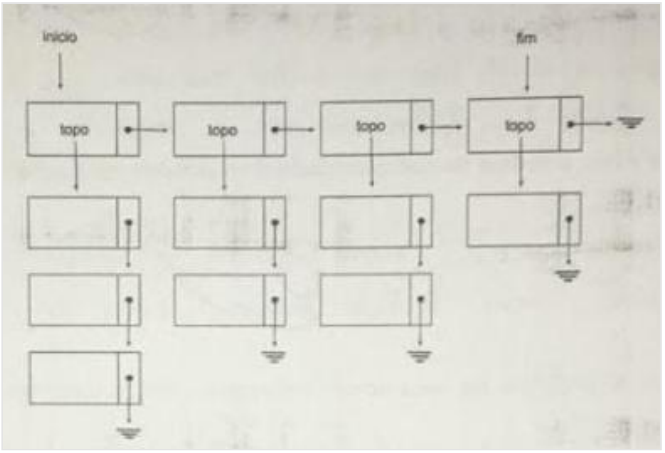


2.4 — Lista de pilhas: maior pilha

Prova II · versão 7 · Questão 1

Considere uma estrutura Lista de Pilhas (ilustrada e codificada abaixo). Crie o método **CelulaLista maiorPilha()** na classe Lista, que retorna a Célula da Lista que aponta para a pilha com o maior numero de elementos. Caso tenham pilhas do mesmo tamanho, retornar a primeira que aparece.

```
class Lista{
   CelulaLista inicio;
   CelulaLista fim;
}
class CelulaLista{
     CelulaPilha topo;
     CelulaLista prox;
}
class CelulaPilha{
     int elemento;
     CelulaPilha prox;
}
```



3. Árvore binária de pesquisa

3.1 — Inserção contando repetições

Prova II · versão 2 · Questão 2

Considere a implementação de uma árvore binária de busca (BST) em que os nós armazenam números inteiros. Na BST vista em sala de aula, não eram aceitos elementos repetidos. Para este exercício, você deve modificar a BST para que, ao inserir um número já existente, em vez de ignorar a inserção, a árvore armazene a quantidade de vezes que cada número foi inserido. Você deve implementar o método ***void inserir(int x)***. Você pode modificar os atributos da árvore. Apresente a complexidade do código em função de theta.

```
class No {
  public int elemento;
  public No esq, dir;
}
```

```
public class Arvore {
    private No raiz;
    public void inserir(int x) {
        // Implemente o método aqui
    }
}
```

3.2 — Ordenar um arranjo usando a árvore

Prova II · versão 5 · Questão 1

Considere o código a seguir de uma árvore binária de pesquisa que armazena números inteiros. Implemente o método estático **ordenar(int[] array)**, que recebe um arranjo de inteiros e ordena os elementos desse arranjo usando, como suporte, uma árvore binária de pesquisa. Considere que apenas os métodos apresentados já estão implementados. Você não pode alterar o protótipo do método **ordenar(int[] array)**. Você tem liberdade para criar métodos que julgar necessários. Apresente a ordem de complexidade do seu método em notação Θ, justificando sua resposta.

```
class No {
    public int elemento;
    public No esq, dir;
}
class Arvore {
    private No raiz;
    public Arvore() {
        raiz = null;
    }
    public void inserir(int x) {
        //metodo ja esta implementado
    }
    public boolean pesquisar(int x) {
        //metodo ja esta implementado
    }
    public int remover(int x) {
        //metodo ja esta implementado
    }
    public static void ordenar(int[] array) {
        //metodo a ser implementado
    }
}
```

3.3 — TreeSort

Prova II · versão 6 · Questão 4

Um algoritmo de ordenação é o TreeSort que insere os elementos do array em uma árvore binária e utiliza um “caminhar” para ordenar os elementos do array. Implemente o TreeSort e faça a análise de complexidade do mesmo.

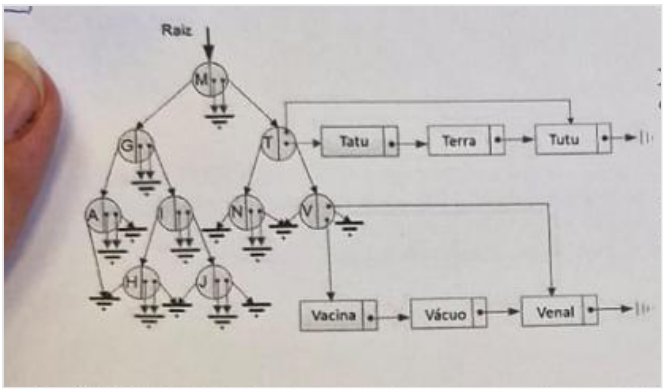
4. Árvore de listas e árvore de árvores (dicionário)

4.1 — Inserir palavra (criando o nó se não existir)

Prova II · versão 3 · Questão 1

Considere uma estrutura de dados que organiza palavras de um dicionário utilizando uma árvore de listas. Nessa estrutura, cada nó representa a primeira letra de um conjunto de palavras, enquanto as palavras são armazenadas em listas encadeadas, mantidas em ordem alfabética. Caso o nó para a letra inicial de uma nova palavra não exista, ele deve ser criado e adicionado à árvore, garantindo a correta ordenação das palavras dentro da lista de cada nó.

Implemente, em Java, o método **inserir** da classe **Dicionario**, responsável por adicionar uma nova palavra a essa estrutura de dados. A seguir, apresenta-se o código básico das classes envolvidas e a figura ilustrando a estrutura de dados:



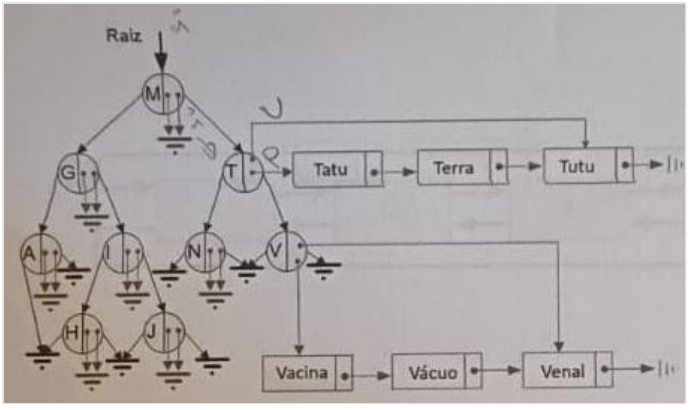

```
class Celula {
    public String palavra;
    public Celula prox;
}
class No {
    public char letra;
    public No esq, dir;
    public Celula primeiro, ultimo;
}
class Dicionario {
    private No raiz;
    public void inserir(String
palavra) {
        //IMPLEMENTAR
    }
}
```

4.2 — Inserir palavra em ordem (lista com célula cabeça)

Prova II · versão 4 · Questão 1

Considere uma estrutura de dados do tipo árvore de listas, conforme ilustrado na figura a seguir, utilizada para armazenar palavras de um dicionário. Os nós da árvore armazenam a letra inicial, enquanto as palavras completas são mantidas nas listas correspondentes em ordem alfabética.

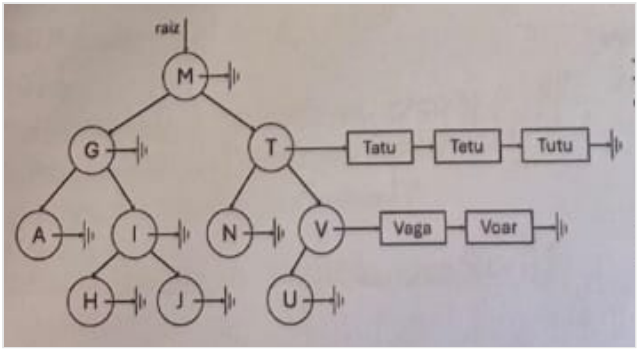
Implemente, em Java, o método public void inserir(String palavra) para inserir uma palavra no Dicionário. O método deve pressupor que todos os nós contendo as letras iniciais já existem na árvore. A inserção deve ser feita de forma ordenada, isto é, deve garantir que a lista onde a palavra for inserida esteja ordenada após a inserção. Na sua solução, considere o código fornecido e assuma que a lista encadeada **possui célula cabeça**, a despeito da figura apresentada. Apresente, em função de θ, a complexidade do seu método.




```
class Celula {
    public String palavra;
    public Celula prox;
}
class No {
    public char letra;
    public No esq, dir;
    public Celula primeiro, ultimo;
}
class Dicionario {
    private No raiz;
    public void inserir(String
palavra) {
        //IMPLEMENTAR
    }
}
```

4.3 — Inserir palavra no final da lista (sem célula cabeça) Prova II · versão 5 · Questão 2

Considere uma estrutura de dados do tipo árvore de listas, conforme mostrado na figura a seguir, utilizada para armazenar palavras de um dicionário. Os nós da árvore guardam a primeira letra da palavra, enquanto as palavras são armazenadas nas listas correspondentes. Implemente, em Java, o método para inserir uma palavra no dicionário. O método deve pressupor que todos os nós com as letras iniciais já existem na árvore. A palavra deve ser inserida no final da lista. Na sua solução, considere o código apresentado e que as listas não possuem célula cabeça. Considere que apenas os métodos apresentados já estão implementados. Você não pode alterar o protótipo do método **inserir(String palavra)**. Você tem liberdade para criar métodos que julgar necessários. Apresente a ordem de complexidade do seu método em notação Θ, justificando sua resposta.




```
class Celula {
    public String palavra;
    public Celula prox;
    public Celula(int p) { palavra = p;
}
}
class No {
    public char letra;
    public No esq, dir;
    public Celula primeiro;
}
class Dicionario {
    private No raiz;
    public void inserir(String palavra)
{
        //metodo a ser implementado
    }
}
```

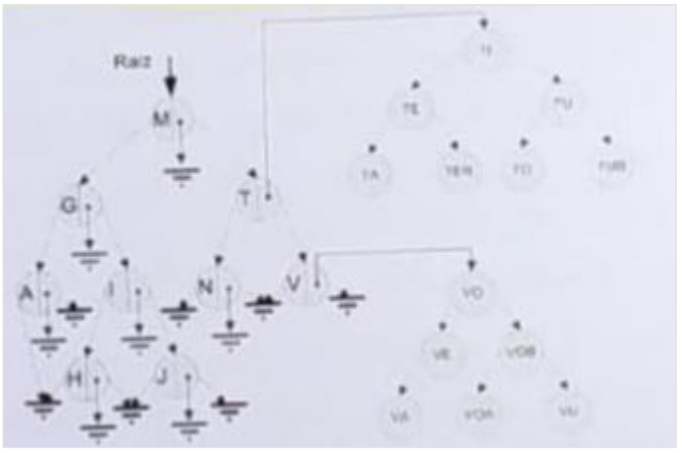
> Assim como na prova, o construtor de Celula aparece como `Celula(int p)`. Na cópia fotografada, alguém corrigiu à mão para `String`.

4.4 — Árvore de árvore: contar palavras por primeira e última letra

Prova II · versão 6 · Questão 1

Na árvore de árvore (ilustrada e codificada abaixo), implemente o método **int contarPalavras(char primeiro, char ultimo)** em Java que recebe dois caracteres e retorna o número de palavras que começam com o caractere primeiro e terminam com o caractere último. Faça a análise de complexidade do seu método.

```
Class ArvoreArvore{
   No raiz;
}
class No{
   char letra;
   No esq, dir;
   No2 raiz;
}
class No2{
   String palavra
   No2 esq, dir;
}
```

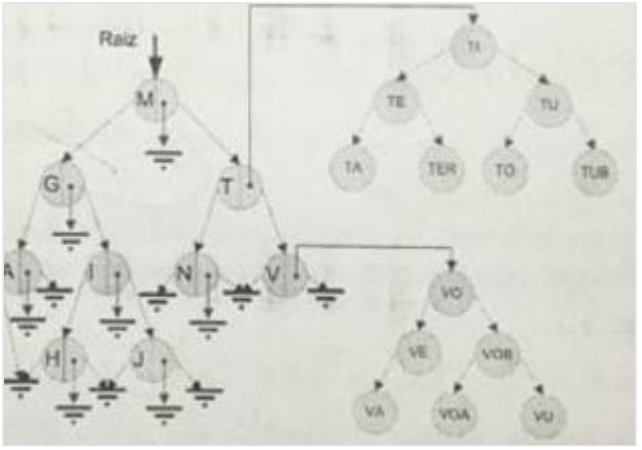


4.5 — Árvore de árvore: contar palavras pelo padrão

Prova II · versão 7 · Questão 2

Na árvore de árvore (ilustrada e codificada abaixo), implemente o método **int contarPalavras(String padrao)** em Java que recebe uma String e retorna o número de palavras que começam com a mesma letra e possuam a mesma quantidade de caracteres. Ex.: Se a palavra passada por parâmetro fosse ”PROVA”, deveriam ser retornadas quantas palavras iniciam com P e possuam 5 caracteres. Obs.: Somente caracteres maiúsculos foram utilizados. Faça a análise de complexidade do seu método.

```
Class ArvoreArvore{
   No raiz;
}
class No{
   char letra;
   No esq, dir;
   No2 raiz;
}
class No2{
   String palavra
   No2 esq, dir;
}
```



5. Matriz flexível

5.1 — Remover a última coluna

Prova II · versão 3 · Questão 2

Considere uma estrutura de dados que representa uma matriz flexível, onde cada célula possui ponteiros para suas células vizinhas (superior, inferior, esquerda e direita). Essa estrutura permite o ajuste dinâmico das dimensões da matriz.

Implemente uma função em C que remova a última coluna dessa matriz flexível, ajustando os ponteiros dos elementos para manter a integridade da estrutura. Abaixo, segue o código básico das estruturas utilizadas:

```
typedef struct Celula {
    int elemento;
    struct Celula *inf, *sup, *esq, *dir;
} Celula;
typedef struct {
    int linhas, colunas;
    Celula *inicio;
} Matriz;
void removerUltimaColuna(Matriz *matriz) {
    //IMPLEMENTAR
}
```

5.2 — Remover a última coluna (com condição de erro)

Prova II · versão 5 · Questão 3

Considere uma estrutura de dados que representa uma matriz flexível, onde cada célula possui ponteiros para suas células vizinhas (superior, inferior, esquerda e direita), conforme apresentado no código em C a seguir. Essa estrutura permite o ajuste dinâmico das dimensões da matriz. Implemente a função **removerUltimaColuna(Matriz* matriz)** que remove a última coluna da matriz, ajustando os ponteiros dos elementos para manter a integridade da estrutura. Considere que apenas os métodos apresentados já estão implementados. Você não pode alterar o protótipo do método **removerUltimaColuna(Matriz* matriz)**. Você tem liberdade para criar métodos que julgar necessários. Uma condição necessária para que a remoção seja realizada é que a matriz resultante tenha no mínimo 1 coluna. Reporte erro caso a condição não seja respeitada. Apresente a ordem de complexidade do seu método em notação Θ, justificando sua resposta.

```
typedef struct Celula {
    int elemento;
    struct Celula *inf, *sup, *esq, *dir;
} Celula;

typedef struct {
    int linhas, colunas;
    Celula *inicio;
} Matriz;

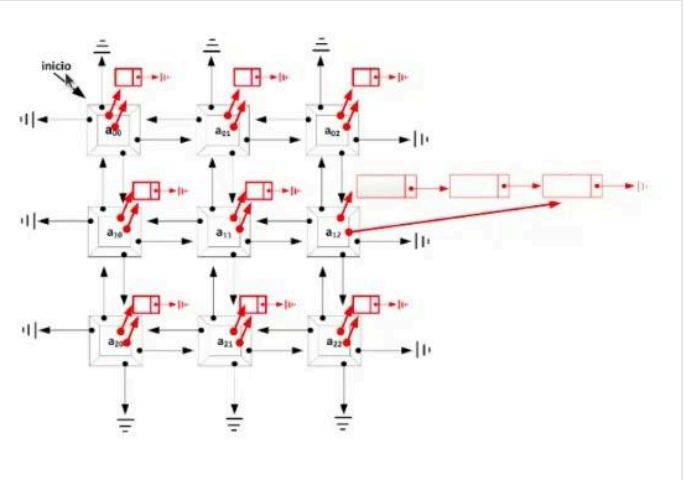
void removerUltimaColuna(Matriz *matriz) {
    //metodo a ser implementado
}
```

5.3 — Diagonal unificada (matriz de listas → lista dupla)

Prova II · versão 2 · Questão 3

Considere a estrutura de classes abaixo, em Java, que representa uma matriz encadeada, onde cada célula da matriz (CelulaMatriz) contém uma lista de células (Celula). A classe Matriz possui um método diagUnificada que precisa ser implementado. Este método deve retornar um ponteiro para uma célula dupla (CelulaDupla) que será o início de uma lista duplamente encadeada contendo todas as células da diagonal principal da matriz (uma única lista duplamente encadeada resultado da concatenação de todas as listas da diagonal principal). Considere que as listas da Matriz possuem Célula Cabeça, que devem ser desconsideradas na lista resultante.

```
class Matriz{
    CelulaMatriz inicio;
    int linha, coluna;
    CelulaDupla diagUnificada(){
    //implemente seu método aqui
    }
}
class CelulaMatriz{
    CelulaMatriz esq, dir, inf, sup;
    Celula inicio, fim;
}
class Celula{
    int elemento;
    Celula prox;
}
class CelulaDupla
    int elemento;
    CelulaDupla prox, ant;
}
```

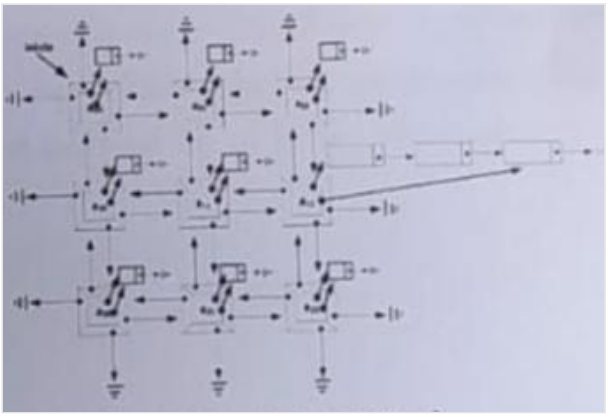


5.4 — Matriz de listas: remover ímpares

Prova II · versão 6 · Questão 3

Seja a matriz de listas em Java apresentada abaixo, faça um método que remove todos os números ímpares existentes nas listas. Faça a análise de complexidade do seu método.

```
class Matriz {
   CelulaMat inicio;
}
class CelulaMat{
   CelulaMat prox, ant, sup, inf;
   Celula primeiro, ultimo;
   public CelulaMat(){
       prox = ant = sup = inf = null;
       primeiro = ultimo = new Celula();
   }
}
class Celula{
   int numero;
   Celula prox;
   public Celula(){ this Celula(0); }
   public Celula(int elemento){
this.numero = elemento; prox = null; }
}
```



6. Estruturas combinadas (árvore + matriz)

6.1 — Elementos da árvore que não estão na matriz

Prova II · versão 4 · Questão 2

Faça o método em Java Celula toLista(No raiz, CelulaMat inicio, int L) que recebe a referência **raiz** de uma árvore binária de pesquisa balanceada com N elementos, cujos nós são do tipo No, e a referência **inicio** de uma matriz flexível quadrada com L linhas e L colunas, conforme a estrutura apresentada em sala de aula. O método deve criar uma lista simplesmente encadeada contendo todos os elementos da árvore que não pertencem à matriz e retornar a referência do nó cabeça da lista criada. Ao final, os elementos da lista devem estar ordenados. Considere que as células da matriz são do tipo CelulaMat e que as células da lista são do tipo Celula, contendo apenas um inteiro e a referência prox, conforme o código básico apresentado abaixo com as estruturas utilizadas. Apresente o método em Java e justifique (i) sua complexidade em função de θ e (ii) a estratégia utilizada para garantir que os elementos da lista estejam ordenados.

```
class Celula {
    int elemento;
    Celula prox;
}
```

```
class CelulaMat {
    int elemento;
    CelulaMat inf, sup,
esq, dir;
}
```

```
class No {
    int elemento;
    No esq, dir;
}
```

```
Celula toLista(No raiz, CelulaMat inicio, int L){
    //IMPLEMENTAR
}
```

6.2 — Números repetidos na árvore e na matriz

Prova II · versão 6 · Questão 2

Dado o código abaixo, faça a função **Celula* encontrarRepetidos(No* raiz, CelulaMatriz* inicio)** em C que recebe o endereço do nó raiz de uma árvore binária e o endereço da primeira célula de uma matriz flexível e retorna o endereço do nó cabeça de uma lista simples ordenada decrescente contendo todos os números que existem tanto na árvore como na matriz flexível. A célula da lista, o nó da árvore e a célula matriz possuem apenas um elemento inteiro e os devidos ponteiros dessas estruturas conforme apresentado na sala de aula. Sabe-se que a árvore não tem elementos repetidos; sabe-se a mesma coisa para a matriz. Faça a análise de complexidade da sua função.

```
typedef Struct{
   int numero;
   Celula *prox;
}Celula;
```

```
typedef Struct{
   int numero;
   CelulaMatriz *prox,
*ant;
   CelulaMatriz *sup,
*inf;
}CelulaMatriz;
```

```
typedef Struct{
   int numero;
   No *esq, *dir;
}No;
```

7. Ordenação

7.1 — Sequência de pivôs do Quicksort

Prova II · versão 1 · Questão 1 (4 pontos)

O algoritmo de ordenação *Quicksort,* apesar de muito utilizado na prática, pode apresentar um comportamento indesejado em algumas situações específicas. O método de ordenação *Quicksort* foi aplicado na ordenação crescente do vetor de inteiros que apresentava, inicialmente, o conteúdo abaixo. Indique toda a **sequência de elementos** do vetor que foram selecionados como **pivô** durante esse processo de ordenação.

23	35	67	2	8	43	97

7.2 — Construção de um max-heap

Prova II · versão 1 · Questão 5 (2 pontos)

O algoritmo de ordenação *Heapsort* utiliza uma estrutura de dados especial para garantir a seleção do maior elemento de um conjunto de dados a baixo custo. Tal estrutura é chamada de *heap*. O *array* de caracteres apresentado abaixo não representa um *max-heap* devidamente restaurado. Apresente, por meio de desenhos, todos os passos necessários na **construção de um *max-heap*** a partir do *array* fornecido. **Dica:** O valor das letras segue a ordem alfabética. Ex: 'A' é menor que 'F'; 'B' é menor que C,....

	F	R	A	M	E	W	O	R	K

7.3 — Juntar dois vetores em ordem crescente

Prova II · versão 7 · Questão 3

Implemente o método **public static int[] vetorOrdenado(int[] vetA, int[] vetB)** em Java que recebe dois arrays de inteiros (tamanhos m e n) e retorna um terceiro array (tamanho m+n). O primeiro array (vetA) possui elementos pares, não primos, ordenados de forma decrescente. O segundo array (vetB) possui elementos ímpares, não primos, ordenados de forma decrescente. O array que será retornado deve possuir os valores dos dois arrays ordenados de forma CRESCENTE. A complexidade do método deve ser O(m+n). A figura abaixo apresenta um exemplo do problema.

vetA	46	38	22	10			
vetB	57	33	21				
vetC	10	21	22	33	38	46	57

8. Verdadeiro ou falso com justificativa

8.1

Prova II · versão 3 · Questão 4

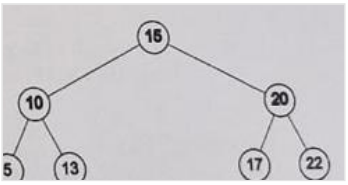
Analise as afirmações a seguir e determine se cada uma é verdadeira ou falsa, apresentando uma justificativa detalhada para sua resposta. Somente as respostas com a justificativa completa serão consideradas.

(a) Você está organizando o catálogo de uma biblioteca e sabe que os livros em sua coleção estão ordenados em ordem crescente por título, com exceção de um único livro fora de posição. Para reorganizar o catálogo completamente em ordem crescente, considerando os algoritmos *merge sort, insertion sort, radix sort, heapsort* e *counting sort,* o algoritmo mais adequado para realizar a tarefa é o *merge sort.*

(b) Você está trabalhando em um dispositivo computacional com restrição de memória RAM e tem urgência em ordenar um conjunto de dados que ocupa um total de 50 MB. Após carregar os dados em memória, o dispositivo apresentou apenas 5 MB de memória livre. Considerando os algoritmos *merge sort, insertion sort, radix sort, heapsort* e *counting sort,* o mais adequado para essa tarefa é o *heapsort.*

(c) O *quicksort* é um dos algoritmos de ordenação mais utilizados devido à sua eficiência em uma ampla variedade de casos. Sua complexidade temporal é $\Theta(n \log n)$ em todos os casos (melhor, médio e pior).

(d) A árvore binária de pesquisa apresentada abaixo possui a seguinte sequência no caminhamento pós-ordem: 15, 10, 5, 13, 20, 17, 22.



(e) Todo *heap* com mais de zero elementos também pode ser considerado uma árvore binária de pesquisa.

8.2

Prova II · versão 5 · Questão 4

Provar ou refutar cada uma das afirmações a seguir, apresentando uma explicação detalhada do motivo pelo qual a afirmação é verdadeira ou falsa. Mesmo que você acredite que a afirmação esteja correta, é necessário justificar com clareza seu raciocínio.

(a) Não é possível desenhar uma árvore binária de busca tendo apenas o caminhamento em Pós-Ordem a seguir:
Pós-ordem: [4, 5, 6, 8, 7, 12, 18, 15, 10, 37, 42, 40, 55, 50, 20]

(b) Os algoritmos Merge Sort e Heap Sort sempre tem complexidade $\Theta(n \log n)$, independentemente do caso (melhor, médio ou pior).

(c) Dado o código abaixo, os valores de c, &c e *c correspondem a endereços de memória.

```
int a = 5;
int *b = &a;
int **c = &b;
```

(d) A inserção e remoção de um elemento no início de uma lista encadeada tem complexidade $\Theta(1)$ apenas quando ela tem célula cabeça.

(e) **Código de Inserção em Lista Duplamente Encadeada**

```
public void insertAtEnd(DoublyListNode head, int value) {
    DoublyListNode newNode = new DoublyListNode(value);
    DoublyListNode temp = head;
    while (temp.next != null) {
        temp = temp.next;
    }
    temp.next = newNode;
    newNode.prev = temp;
}
```

O código acima insere um novo elemento no final de uma lista duplamente encadeada com complexidade $\Theta(n)$.

> A foto desta página está inclinada e corta a margem direita. As palavras finais de algumas linhas foram completadas a partir de outra foto em que a mesma página aparece ao fundo.

Apêndice — Prova I (outro conteúdo, não cai na Prova II)

> Uma das fotos enviadas é de uma **Prova I**. Ela foi transcrita aqui só para registro.

A.1 — Contagem de operações

Considere o código abaixo. Determine os itens que se seguem:

```
int x = 0;
for(int i = 15; i < n; i++){
   for(int j = 0; j < 3; j++) x+= a * 2;
   for(int k = i; k < n; k++) {
      x+= b * 5;
      x+= x * 10;
   }
   x+= c * 4;
}
```

(a) a função de complexidade usando a notação Σ para o número de multiplicações;
(b) a fórmula fechada da função de complexidade e a ordem de complexidade usando a notação Θ.

A.2 — Somatório e indução

Encontre a fórmula fechada para o somatório a seguir e, em seguida, faça a prova por indução matemática.

$$\Sigma_{i=1}^{n} [(10i + 2)^2 - (10i + 1)^2]$$

A.3 — Fila de prioridade

Considere a classe em Java denominada FilaPrioridade, que representa uma estrutura de dados do tipo Fila de Prioridade e que utiliza o conceito de máxima prioridade para determinar a ordem de remoção dos elementos. Elementos de maior prioridade devem ser removidos antes dos elementos de menor prioridade. Essa fila será utilizada para o atendimento de pacientes médicos, onde os pacientes são representados por nome e prioridade de atendimento.

```
class Paciente {
    public String nome;
    public int prioridade;
}
class FilaPrioridade {
    private Paciente[] itens;
    private int n;
    public FilaPrioridade(int tam) { //IMPLEMENTAR }
    public void inserir(Paciente p) { //IMPLEMENTAR }
    public Paciente remover() { //IMPLEMENTAR }
}
```

Implemente o construtor e os métodos inserir e remover da fila. Considerações: o construtor gera uma fila que é capaz de armazenar, no máximo, "tam" pacientes; a inserção insere um paciente na fila, respeitando a ordem de prioridade; a remoção remove e retorna o paciente de maior prioridade da fila e, caso exista mais de um paciente com a mesma prioridade, remove o que foi inserido primeiro; os casos de estouro e underflow (remoção em fila vazia) devem ser detectados.

A.4 — Seleção modificada

Modifique o algoritmo de ordenação Seleção para que, a cada iteração, seja selecionado tanto o menor quanto o maior elemento da porção não ordenada do arranjo. O menor elemento será movido para o início e o maior elemento será movido para o final da lista. Ao final do processo, o array estará ordenado de forma crescente. Os seguintes itens devem ser apresentados:

(a) Apresente o código em C-Like para essa versão modificada do Seleção.
(b) Explique como essa modificação impacta a complexidade temporal do algoritmo, apresentando a nova complexidade.
(c) Considere o array [29, 10, 14, 37, 13, 5, 26]. Mostre o estado do array após cada iteração usando o algoritmo modificado.